Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №1

з дисципліни
«Проєктування складних систем»

на тему

«Дослідження наслідків роботи протоколу когерентності кешів MESIF»

Виконав:

студент групи ІМ-21

Сірик Максим

Варіант 20

Перевірив:

Долголенко Олександр

Київ 2026

Мета роботи: Дослідити наслідки роботи протоколу когерентності кешів MESIF

Постановка завдання

Розробіть два екземпляра свого варіанта багатопотокової програми на Java. Перший екземпляр багатопотокової програми розробіть, за додержанням принципу локалізації. При цьому як скалярні так і векторні змінні не будуть розділюватися між потоками – механізм когерентності кешів MESIF працювати не буде. Другий екземпляр багатопотокової програми розробіть за використанням скалярних та векторних глобальних змінних та механізмів синхронізації потоків при зверненні до цих змінних – пошук актуального значення змінної при цьому буде відбуватися, відповідно до механізму когерентності кешів MESIF, у кешах L1D всіх ядер мікропроцесора. Обидва варіанти програми повинні складалися з однакової кількості потоків, але звертатися до глобальних даних по різному. В одному з них повинна бути як можна менша кількість звернень до глобальних даних, а в іншому - як можна більша (щоб на час виконання не впливала різна кількість потоків в варіантах, а тільки впливав MESIF). Побудуйте порівняльні графіки й зробіть висновки по відносній швидкодії цих екземплярів програми.

- ☑ ~~програма, що використовує локальні змінні~~
- ☑ ~~програма, що звертається до глобальних змінних~~
- ☑ ~~розмірність векторів та матриць: від 500 до 1000~~
- ☑ ~~всі початкові дані – додатні числа з плаваючою крапкою;~~
- ☑ ~~порядки елементів матриць та векторів повинні бути різними;~~
- ☑ ~~початкові дані збережіть в файлі і вводити дані з файлу;~~
- ☑ ~~організувати вивід результатів обчислень функцій в файл;~~
- ☑ ~~для підвищення точності обчислень з плаваючою крапкою кожне додавання довгої послідовності чисел виконуйте з використанням алгоритму типу Kahan, або перед виконанням додавання послідовності чисел спершу її відсортуйте по зростанню, а потім підсумовуйте, починаючи з найменших чисел;~~
- ☑ ~~з використанням засобів синхронізації розробіть паралельну програму на Java для обчислення двох математичних функцій;~~

- ☑ ~~в першій роботі, щоб побачити вплив механізму когерентності кешів MESIF на зменшення часу виконання 2 варіанту відносно часу виконання 1 варіанта потрібно значно змінювати розмірність оброблюваних матриць та векторів;~~

Ваш варіант математичних функцій для обчислення двох перших лабораторних робіт визначається Вашим номером в списку групи (20) (де: MA і т. п. – матриця, розміром nxn; A і т. п. – вектор, розміром n; a і т. п. – скаляр):

ВАРІАНТ 20

MG = MB*MT + MC*(MB*MM+MT);

B = MT*D - min(D)*C.

Виконання завдання

Перевірка коректності обчислень (N=4)

Калькулятор

$A \cdot B + C(A \cdot D + B)$	$= \begin{bmatrix} 5.31381 & 3.13003 & 5.24965 & 3.51278 \\ 4.81694 & 2.70965 & 4.44099 & 3.15715 \\ 3.60603 & 2.28417 & 3.80512 & 2.47784 \\ 6.56986 & 3.9651 & 6.29817 & 4.426 \end{bmatrix}$
$B \cdot F - 0.0242656 \cdot E$	$= \begin{bmatrix} 0.869251 \\ 1.43038 \\ 0.490417 \\ 0.791108 \end{bmatrix}$

Без підвищення точності

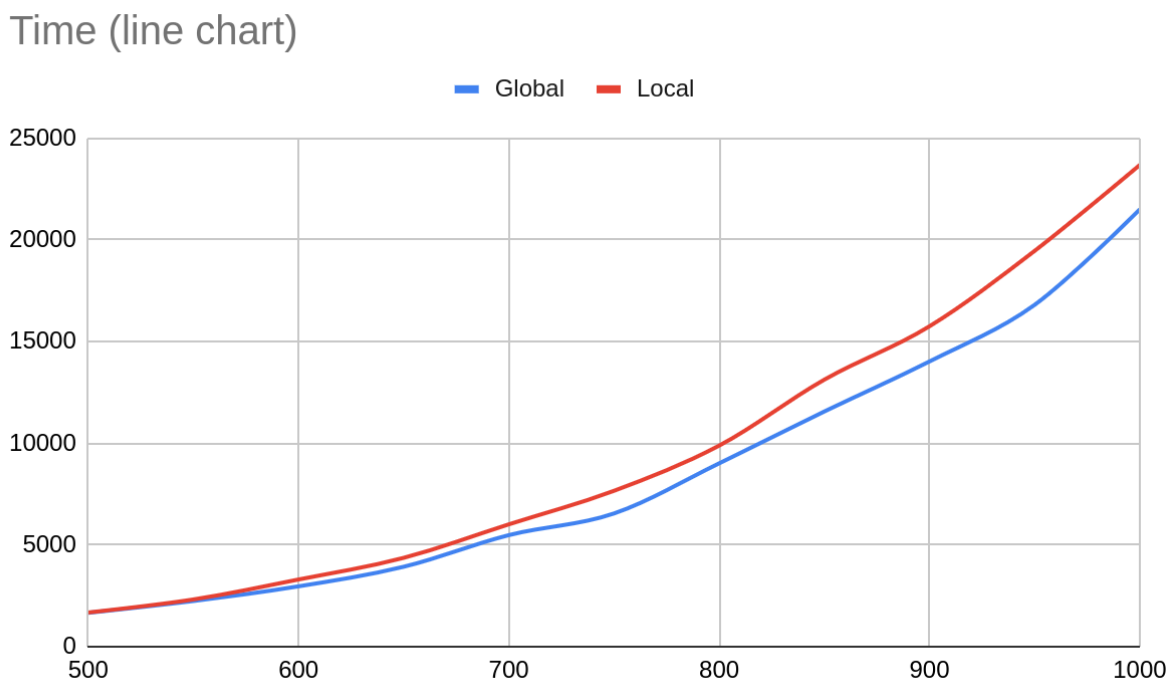
```
Time: 3 ms

MG:
5.313807007245712 3.1300314193144025 5.249645844576559 3.5127756351508266
4.816944296274034 2.709645275523285 4.440991804476623 3.1571464891722716
3.606026217971074 2.2841728759652504 3.8051164864040867 2.47783618496378
6.569864562335635 3.965101841217651 6.298168519696567 4.425999300193713

B:
0.8692505189207308
1.4303829723242054
0.49041728332527745
0.7911078950395201
```

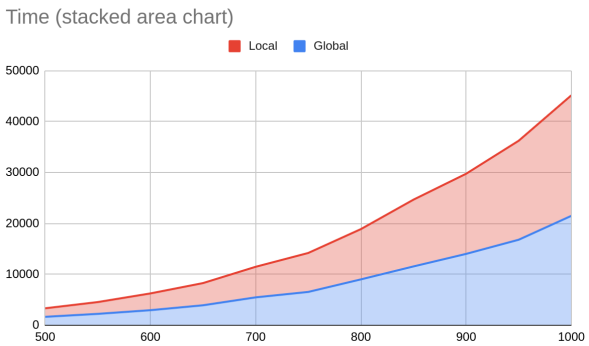
З підвищенням точності

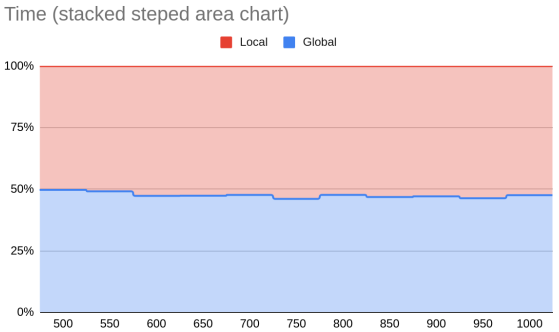
```
Time: 12 ms

MG:
5.313807007245712 3.1300314193144025 5.249645844576559 3.5127756351508266
4.816944296274033 2.709645275523285 4.440991804476623 3.157146489172272
3.606026217971074 2.2841728759652504 3.8051164864040867 2.47783618496378
6.569864562335634 3.965101841217651 6.298168519696567 4.425999300193713

B:
0.8692505189207308
1.4303829723242056
0.49041728332527745
0.7911078950395201
```

Графіки

ПСС_ЛР1_ІМ-21_Сірик_Максим_Олександрович







N	500	550	600	650	700	750	800	850	900	950	1000
Час виконяння з локальними змінними	1505	2012	3003	4099	5634	6660	9521	15636	20077	22899	28224
	1637	2137	3017	4099	7124	8141	9682	12519	14430	18760	22343
	1673	2549	3634	5012	5730	7615	9937	12855	14916	17608	22649
	1735	2456	3450	4323	5715	7926	9981	12395	15473	18281	22581
	1677	2516	3125	4316	5908	7464	10053	12670	14929	20883	22303

	1841	2239	3565	4332	5972	8160	10120	12684	14666	18319	23994
Average	1678	2318.166667	3299	4363.5	6013.833333	7661	9882.333333	13126.5	15748.5	19458.33333	23682.33333
Час виконяння з глобальними змінними	2172	2258	2982	3460	5652	5871	8702	11106	13204	15776	21065
	1503	1880	2582	3373	4984	5878	8729	11142	14130	15846	21194
	1363	2547	2856	4425	5255	6919	9251	11541	14156	17250	22121
	1569	2374	2900	3989	5731	6734	9343	11914	14287	17366	20652
	1793	2056	3434	4015	5585	6847	8996	11688	14258	17240	22373
	1558	2324	3009	4286	5669	7022	9026	11951	14076	17331	21566
Average	1659.666667	2239.833333	2960.5	3924.666667	5479.333333	6545.166667	9007.833333	11557	14018.5	16801.5	21495.16667

Лістинг

```
import kotlin.Triple;

import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Random;
import java.util.Scanner;
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;
import java.util.function.Function;
import java.util.function.Supplier;

interface IMatrix {
    /**
     * ```
     * | a11 a12 a13 |   | b11 b12 |   | c11 c12 |
     * | a21 a22 a23 | * | b21 b22 | =
     * | a31 a32 a33 |   | b31 b32 |
     * ```
     * mXn * nXp = mXp
     */
    static Matrix multiply(IMatrix MA, IMatrix MB, int rowStart, int
rowEnd, int colStart, int colEnd) {
        if (rowEnd == -1) rowEnd = MA.size(0) - 1;
        if (colEnd == -1) colEnd = MB.size(1) - 1;

        int m = MA.size(0);
        int n = MA.size(1);
        int p = MA.size(1);

        assert n == MB.size(0) : "Can't multiply matrices";
        assert rowEnd >= rowStart : "rowEnd must be greater than or
equal to rowStart";
        assert rowEnd < m : "rowEnd must be less than the number of rows
in MA";
        assert colEnd >= colStart : "colEnd must be greater than or
equal to colStart";
        assert colEnd < p : "colEnd must be less than the number of
columns in MB";

        int numRows = rowEnd - rowStart + 1;
```

```
        int numCols = colEnd - colStart + 1;
        double[][] result = new double[numRows][numCols];

        for (int i = rowStart; i <= rowEnd; i++) {
            for (int k = colStart; k <= colEnd; k++) {
                final int ii = i;
                final int kk = k;
                var sum = Summator.kahanSum(
                        0, n - 1,
                        (j) -> MA.get(ii, j) * MB.get(j, kk)
                );
                result[i - rowStart][k - colStart] = sum;
// Equivalent to:
//                for (int j = 0; j < n; j++) {
//                    result[i - rowStart][k - colStart] += MA.get(i, j)
* MB.get(j, k);
//                }
            }
        }

        return new Matrix(result);
    }

    static Matrix multiplyScalar(IMatrix MA, double scalar, int
rowStart, int rowEnd, int colStart, int colEnd) {
        if (rowEnd == -1) rowEnd = MA.size(0) - 1;
        if (colEnd == -1) colEnd = MA.size(1) - 1;

        assert rowEnd >= rowStart : "rowEnd must be greater than or
equal to rowStart";
        assert rowEnd < MA.size(0) : "rowEnd must be less than the
number of rows in MA";
        assert colEnd >= colStart : "colEnd must be greater than or
equal to colStart";
        assert colEnd < MA.size(1) : "colEnd must be less than the
number of columns in MA";

        int m = rowEnd - rowStart + 1;
        int n = colEnd - colStart + 1;

        double[][] result = new double[m][n];

        for (int i = rowStart; i <= rowEnd; i++) {
            for (int j = colStart; j <= colEnd; j++) {
                result[i - rowStart][j - colStart] = MA.get(i, j) *
scalar;
```

```
            }
        }

        return new Matrix(result);
    }

    static Matrix sum(IMatrix MA, IMatrix MB) {
        return sum(MA, MB, 1);
    }

    static Matrix diff(IMatrix MA, IMatrix MB) {
        return sum(MA, MB, -1);
    }

    private static Matrix sum(IMatrix MA, IMatrix MB, int sign) {
        sign = Integer.signum(sign);

        int m = MA.size(0);
        int n = MA.size(1);

        assert m == MB.size(0) : "Can't sum matrices";
        assert n == MB.size(1) : "Can't sum matrices";

        double[][] result = new double[m][n];

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                double[] values = {MA.get(i, j), sign * MB.get(i, j)};
                result[i][j] = Arrays.stream(values).sorted().sum();
            }
        }

        return new Matrix(result);
    }

    static double min(IMatrix A, int rowStart, int rowEnd, int colStart,
int colEnd) {
        if (rowEnd == -1) rowEnd = A.size(0) - 1;
        if (colEnd == -1) colEnd = A.size(1) - 1;

        assert rowEnd >= rowStart : "rowEnd must be greater than or
equal to rowStart";
        assert rowEnd < A.size(0) : "rowEnd must be less than the number
of rows in A";
        assert colEnd >= colStart : "colEnd must be greater than or
equal to colStart";
```

```
        assert colEnd < A.size(1) : "colEnd must be less than the number
of columns in A";

        double minValue = Double.MAX_VALUE;

        for (int i = rowStart; i <= rowEnd; i++) {
            for (int j = colStart; j <= colEnd; j++) {
                double value = A.get(i, j);
                minValue = Math.min(minValue, value);
            }
        }

        return minValue;
    }

    double get(int i, int j);

    int size(int dim);

    class TransposeView implements IMatrix {
        private final IMatrix MA;

        public TransposeView(IMatrix MA) {
            this.MA = MA;
        }

        @Override
        public double get(int i, int j) {
            return MA.get(j, i);
        }

        @Override
        public int size(int dim) {
            if (dim == 0) return MA.size(1);
            else if (dim == 1) return MA.size(0);
            else
                throw new IllegalArgumentException("Invalid dimension: "
+ dim);
        }
    }

    class OffsetView implements IMatrix {
        private final IMatrix MA;
        private final int colOffset;
        private final int colLen;
```

```java
        public OffsetView(IMatrix MA, int colOffset, int colLen) {
            this.MA = MA;
            this.colOffset = colOffset;
            this.colLen = colLen;
        }

        @Override
        public double get(int i, int j) {
            return MA.get(i, j + colOffset);
        }

        @Override
        public int size(int dim) {
            if (dim == 0) return MA.size(0);
            else if (dim == 1) return this.colLen;
            else
                throw new IllegalArgumentException("Invalid dimension: "
+ dim);
        }
    }
}

class Summator {
    static double kahanSum(int start, int end, Function<Integer, Double>
doubleProvider) {
        double sum = 0;
        double err = 0;
        for (int i = start; i <= end; i++) {
            var corrected = doubleProvider.apply(i) - err;
            var newSum = sum + corrected;
            err = (newSum - sum) - corrected;
            sum = newSum;
        }
        return sum;
    }
}

class Matrix implements IMatrix {
    private final double[][] MA;

    public Matrix(double[][] MA) {
        this.MA = MA;
    }

    static Matrix fromVector(double[] vector) {
        var matrix = new double[1][vector.length];
```

```
        matrix[0] = vector;
        return new Matrix(matrix);
    }

    public IMatrix getTransposeView() {
        return new TransposeView(this);
    }

    public IMatrix getOffsetView(int start, int end) {
        return new OffsetView(this, start, end - start + 1);
    }

    public Matrix copy() {
        var copy = new double[MA.length][MA[0].length];
        for (int i = 0; i < MA.length; i++) {
            copy[i] = Arrays.copyOf(MA[i], MA[i].length);
        }
        return new Matrix(copy);
    }

    @Override
    public double get(int i, int j) {
        return MA[i][j];
    }

    @Override
    public int size(int dim) {
        if (dim == 0) return MA.length;
        else if (dim == 1) return MA[0].length;
        else throw new IllegalArgumentException("Invalid dimension: " +
dim);
    }
}

record Data(
        Matrix MB,
        Matrix MT,
        Matrix MC,
        Matrix MM,

        Matrix D,
        Matrix C
) {
    Data deepCopy() {
        return new Data(
                MB.copy(),
```

```
                MT.copy(),
                MC.copy(),
                MM.copy(),
                D.copy(),
                C.copy()
        );
    }
}

record Conf(
        int N,
        int amountOfThreads
) {
    public Conf {
        assert N % amountOfThreads == 0 : "N must be divisible by
amountOfThreads";
    }
}

class FileUtils {
    static String baseDir = "data/";

    static String getInputFileName(int N) {
        return N + "_input.txt";
    }

    static String getOutputFileName(int N) {
        return N + "_output.txt";
    }

    public static void writeResult(int N, Triple<Long, Matrix[],
Matrix[]> result) {
        long time = result.getFirst();
        Matrix[] MG_i = result.getSecond();
        Matrix[] B_i = result.getThird();
        var file = new File(baseDir, getOutputFileName(N));
        try (var fileWriter = new FileWriter(file)) {
            fileWriter.write("Time: " + time + " ms\n");
            fileWriter.write("\nMG:\n");
            writeMatrixColParts(fileWriter, MG_i);
            fileWriter.write("\nB:\n");
            writeMatrixRowParts(fileWriter, B_i);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }
```

```java
    private static void writeMatrixColParts(FileWriter fileWriter,
Matrix[] matrices) throws IOException {
        for (int k = 0; k < matrices[0].size(0); k++) {
            for (Matrix matrix : matrices) {
                for (int j = 0; j < matrix.size(1); j++) {
                    var result = matrix.get(k, j) + " ";
                    fileWriter.write(result);
                }
            }
            fileWriter.write("\n");
        }
    }

    private static void writeMatrixRowParts(FileWriter fileWriter,
Matrix[] matrices) throws IOException {
        for (Matrix matrix : matrices) {
            for (int k = 0; k < matrices[0].size(0); k++) {
                for (int j = 0; j < matrix.size(1); j++) {
                    var result = matrix.get(k, j) + " ";
                    fileWriter.write(result);
                }
                fileWriter.write("\n");
            }
        }
    }

    public static Data readData(int N) {
        var file = new File(baseDir, getInputFileName(N));
        if (!file.exists()) {
            generateRandomData(N, file);
        }

        try (var scanner = new Scanner(file)) {
            double[][] MB = new double[N][N];
            double[][] MT = new double[N][N];
            double[][] MC = new double[N][N];
            double[][] MM = new double[N][N];
            double[] C = new double[N];
            double[] D = new double[N];

            for (var matrix : new double[][][]{MB, MT, MC, MM}) {
                for (int i = 0; i < N; i++) {
                    for (int j = 0; j < N; j++) {
                        matrix[i][j] = scanner.nextDouble();
                    }
```

```
                }
            }

            for (var vector : new double[][]{C, D}) {
                for (int i = 0; i < N; i++) {
                    vector[i] = scanner.nextDouble();
                }
            }

            return new Data(
                    new Matrix(MB),
                    new Matrix(MT),
                    new Matrix(MC),
                    new Matrix(MM),
                    Matrix.fromVector(D),
                    Matrix.fromVector(C)
            );
        } catch (FileNotFoundException e) {
            throw new RuntimeException(e);
        }
    }

    private static void generateRandomData(int N, File file) {
        var random = new Random(42);
        var getDigit = (Supplier<Double>) () -> {
            int digits = random.nextInt(10) + 1;
            double result = Math.round(random.nextDouble() *
Math.pow(10, digits)) / Math.pow(10, digits);
            return result;
        };

        try (var fileWriter = new FileWriter(file)) {
            for (int t = 0; t < 4; t++) {
                for (int i = 0; i < N; i++) {
                    for (int j = 0; j < N; j++) {
                        var num = getDigit.get();
                        fileWriter.write(num + " ");
                    }
                    fileWriter.write("\n");
                }
                fileWriter.write("\n");
            }

            for (int t = 0; t < 2; t++) {
                for (int i = 0; i < N; i++) {
                    var num = getDigit.get();
```

```
                    fileWriter.write(num + "\n");
                }
                fileWriter.write("\n");
            }
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }
}

/**
 *
=========================================================================
=====
 *
=========================================================================
=====
 *
=========================================================================
=====
 */
public class Main {
    public static void main(String[] args) throws InterruptedException {
        for (int i = 0; i < 5; i++) {
            var times = new ArrayList<Long>();
            for (int N = 500; N <= 1000; N += 50) {
                var conf = new Conf(N, Thread.activeCount());
                var data = FileUtils.readData(conf.N());
                var result = Runner.run(conf, data, false);
//                var result = Runner.run(conf, data, true);
                FileUtils.writeResult(conf.N(), result);
                System.out.println("N: " + conf.N() + ", time: " +
result.getFirst() + " ms");
                times.add(result.getFirst());
            }

            System.out.println(times);
        }
    }
}


class Runner {

    public static Triple<Long, Matrix[], Matrix[]> run(Conf conf, Data
data, boolean isUsingGlobals) throws InterruptedException {
```

```java
        var ts = new T[conf.amountOfThreads()];
        var threads = new Thread[conf.amountOfThreads()];
        var context = new MinContext(conf.amountOfThreads());
        for (int i = 0; i < conf.amountOfThreads(); i++) {
            int diapason = conf.N() / conf.amountOfThreads();
            int rowStart = i * diapason;
            int rowEnd = (i + 1) * diapason - 1;
            Data fullCopy = isUsingGlobals ? data : data.deepCopy();
            ts[i] = new T(fullCopy, rowStart, rowEnd, context);
            threads[i] = new Thread(ts[i]);
        }

        long startTime = System.currentTimeMillis();

        for (var thread : threads) {
            thread.start();
        }

        for (var thread : threads) {
            thread.join();
        }

        long endTime = System.currentTimeMillis();
        long elapsedTime = endTime - startTime;

        Matrix[] result1 = new Matrix[conf.amountOfThreads()];
        Matrix[] result2 = new Matrix[conf.amountOfThreads()];
        for (int i = 0; i < ts.length; i++) {
            result1[i] = ts[i].MG_i;
            result2[i] = ts[i].B_i;
        }

        return new Triple<>(elapsedTime, result1, result2);
    }

    private static class T implements Runnable {
        private final int start;
        private final int end;
        private final Data data;
        private final MinContext minContext;
        Matrix MG_i;
        Matrix B_i;

        public T(Data data, int start, int end, MinContext minContext) {
            this.data = data;
            this.start = start;
```

```
            this.end = end;
            this.minContext = minContext;
        }

        private void calc1() {
            var MB_MM_i = IMatrix.multiply(
                    data.MB(),
                    data.MM(),
                    0, -1,
                    start, end
            );
            var MB_MM_MT_i = IMatrix.sum(
                    MB_MM_i,
                    data.MT().getOffsetView(start, end)
            );
            var MC_MB_MM_MT_i =
                    IMatrix.multiply(
                            data.MC(),
                            MB_MM_MT_i,
                            0, -1,
                            0, -1
                    );
            var MB_MT_i = IMatrix.multiply(
                    data.MB(),
                    data.MT(),
                    0, -1,
                    start, end
            );
            MG_i = IMatrix.sum(
                    MB_MT_i,
                    MC_MB_MM_MT_i
            );
        }

        private void calc2() throws BrokenBarrierException,
InterruptedException {
            var min_D_i = IMatrix.min(data.D(), 0, -1, start, end);
            var min_D = minContext.compareAndGet(min_D_i);
            var min_d_C = IMatrix.multiplyScalar(data.C(), min_D, 0, -1,
start, end);
            var MT_D_i = IMatrix.multiply(
                    data.MT(),
                    data.D().getTransposeView(),
                    start, end,
                    0, -1
            );
```

```java
            B_i = IMatrix.diff(
                    MT_D_i,
                    min_d_C.getTransposeView()
            );
        }

        @Override
        public void run() {
            try {
                calc1();
                calc2();
            } catch (Exception e) {
                throw new RuntimeException(e);
            }
        }
    }
}

class MinContext {
    private final CyclicBarrier cb;
    private double min = Double.MAX_VALUE;

    public MinContext(int amountOfThreads) {
        cb = new CyclicBarrier(amountOfThreads);
    }

    public double compareAndGet(double min) throws
BrokenBarrierException, InterruptedException {
        synchronized (this) {
            this.min = Math.min(this.min, min);
        }
        cb.await();
        return this.min;
    }
}
```

Висновок

У лабораторній роботі було реалізовано та порівняно два багатопотокові варіанти програми на Java для обчислення заданих матрично-векторних функцій при розмірностях N=500..1000: варіант із локалізацією даних (кожен потік працює зі своєю копією структур) та варіант зі спільними (глобальними) даними. Для підвищення точності підсумовування застосовано алгоритм Kahan.

За результатами експерименту середній час виконання варіанту зі спільними (глобальними) змінними є стабільно меншим за варіант із локалізацією даних для всіх досліджених N, перевага незначна (становить лише кілька мілісекунд у межах загального часу) для N < 900, і поступово збільшується (t > 1s) для N=950, 1000. Також слід врахувати, що вимірювання часу починалося після етапу підготовки/копіювання даних, тобто накладні витрати на deepCopy() у “локальному” варіанті **не враховані**. Відтак отримана різниця відображає переважно витрати та ефекти, що виникають під час власне обчислень і синхронізації.

Важливим спостереженням є те, що механізми синхронізації задіяні в обох реалізаціях (зокрема, для обчислення глобального мінімуму використано synchronized та CyclicBarrier). Через це обидва варіанти мають накладні витрати на узгодження виконання потоків. Основний час роботи програми визначається обсягом матричних множень і проходів по даних, тому навіть наявні затрати на когерентність/обмін кеш-лініями дають виграш, не такий різкий як я очукував.

Отже, поставлену мету досягнуто: реалізовано два варіанти паралельної програми та виконано експериментальне порівняння. Зроблено висновок, що в обраній реалізації та діапазоні розмірностей ефект MESIF не є суттєвим, а результати більше залежать від загальної обчислювальної складності, накладних витрат синхронізації та варіативності виконання в середовищі ОС/JVM.